MODULE

20

Capstone: budget

One small project built like a professional one, with every core module working together.

YOU WILL LEARN

The project	`budget_project/`
Architecture	`budget_project/README.md`
Service + repository	`src/budget/service.py`
CLI + API	`cli.py` · `api.py`
Tests	`budget_project/tests/`
Challenges	`ex20_capstone_challenges.md`

WHY ARCHITECTURE

Why structure matters more than clever code

THE PROBLEM

Version 1 put SQL, input parsing and business rules in one 300-line function. Adding an API meant **copy-pasting** it all. Testing it needed a real database.

THE IDEA

Split the code into layers. Each layer has one job and talks only to the layer below.

PICTURE IT ARCHITECTURE

Clean layers

🏢 Like a **restaurant**: waiters (CLI, API) take orders, the kitchen (service) cooks by the rules, the store room (repository) holds the ingredients. Waiters never walk into the store room.



The service depends on a **Protocol**, not on SQLite: that's dependency injection.

TRY IT SERVICE + REPOSITORY

The core pattern in 20 lines

```python
from collections import defaultdict
from dataclasses import dataclass

@dataclass(frozen=True)
class Expense:
    amount: float
    category: str
class MemoryRepo:                          # a fake for tests
    def __init__(self): self.items = []
    def add(self, e): self.items.append(e)
    def all(self): return list(self.items)
class BudgetService:
    def __init__(self, repo): self.repo = repo      # injected
    def add(self, amount, category):
        if amount <= 0:
            raise ValueError("amount must be positive")
        self.repo.add(Expense(amount, category))
    def summary(self):
        totals = defaultdict(float)
        for e in self.repo.all():
            totals[e.category] += e.amount
        return dict(sorted(totals.items(), key=lambda kv: -kv[1]))
svc = BudgetService(MemoryRepo())
svc.add(320, "food"); svc.add(80, "transport"); svc.add(45, "food")
print(svc.summary())
```

OUTPUT

```
{'food': 365.0, 'transport': 80.0}
```

1. The service never sees SQL.
2. Tests pass a **MemoryRepo**; production passes a **SqliteRepo**.
3. CLI and API both call the **same** service.

STEP BY STEP MODULE MAP

Where every module shows up

MODULE	IN THE CAPSTONE
8 Pythonic · 9 Data structures	comprehensions, `sorted(key=)`, `@dataclass(frozen=True)`, `defaultdict`
10 Functions · 12 OOP	FastAPI decorators, `Protocol`, dependency injection
13 Errors & logging	`BudgetError`, `raise ... from`, `getLogger(__name__)`
14 Packaging · 15 Typing	src layout, `pyproject.toml`, the `budget` command, `mypy --strict`
16 Testing	fixtures, parametrize, `TestClient` API tests
18 Data & APIs	SQLite repository, FastAPI app

> Read **budget_project/README.md** first, then the code top-down: cli → service → repository.

IN REAL JOBS CAPSTONE

Run it like a real project

Install and use the CLI

```
pip install -e ".[dev]"
budget add 320 food "groceries"
budget summary --month 2026-11
```

Serve the API

```
uvicorn budget.api:app --reload
# open http://127.0.0.1:8000/docs
```

The quality gate

```
pytest -q
mypy --strict src
ruff check .
```

Show it off

```
README + architecture diagram
+ tests passing = a portfolio project
```

INTERVIEW SAY THIS IN THE INTERVIEW

“Walk me through the architecture of a project you built.”

- Start with the **goal**: ”track expenses from a CLI and a REST API”.
- Draw the **layers**: interface (CLI/API) → service (rules) → repository (SQL) → DB.
- Explain one **design decision** and its trade-off: a Protocol + DI made the service testable without a DB.
- Mention **quality**: tests with fixtures, mypy --strict, ruff, CI.
- End with what you’d **improve next**: Postgres, auth, Docker, monitoring.

> **Follow-up they ask next:** ”Why the repository pattern?” (all SQL in one place, swappable storage, easy fakes in tests, the service stays pure).

RECAP REMEMBER THESE

Core track done: what you can do now

1. Write **Pythonic**, typed, tested code.
2. Pick the right **data structure** and explain its Big-O.
3. Use closures, decorators, generators and context managers.
4. Design with **layers**, Protocols and dependency injection.
5. Package a CLI, call APIs, use SQL, serve FastAPI.
6. Measure and fix performance, and use concurrency well.

PRACTICE NOW

Do the challenges in exercises/ex20_capstone_challenges.md, then start the web track (Module 21).